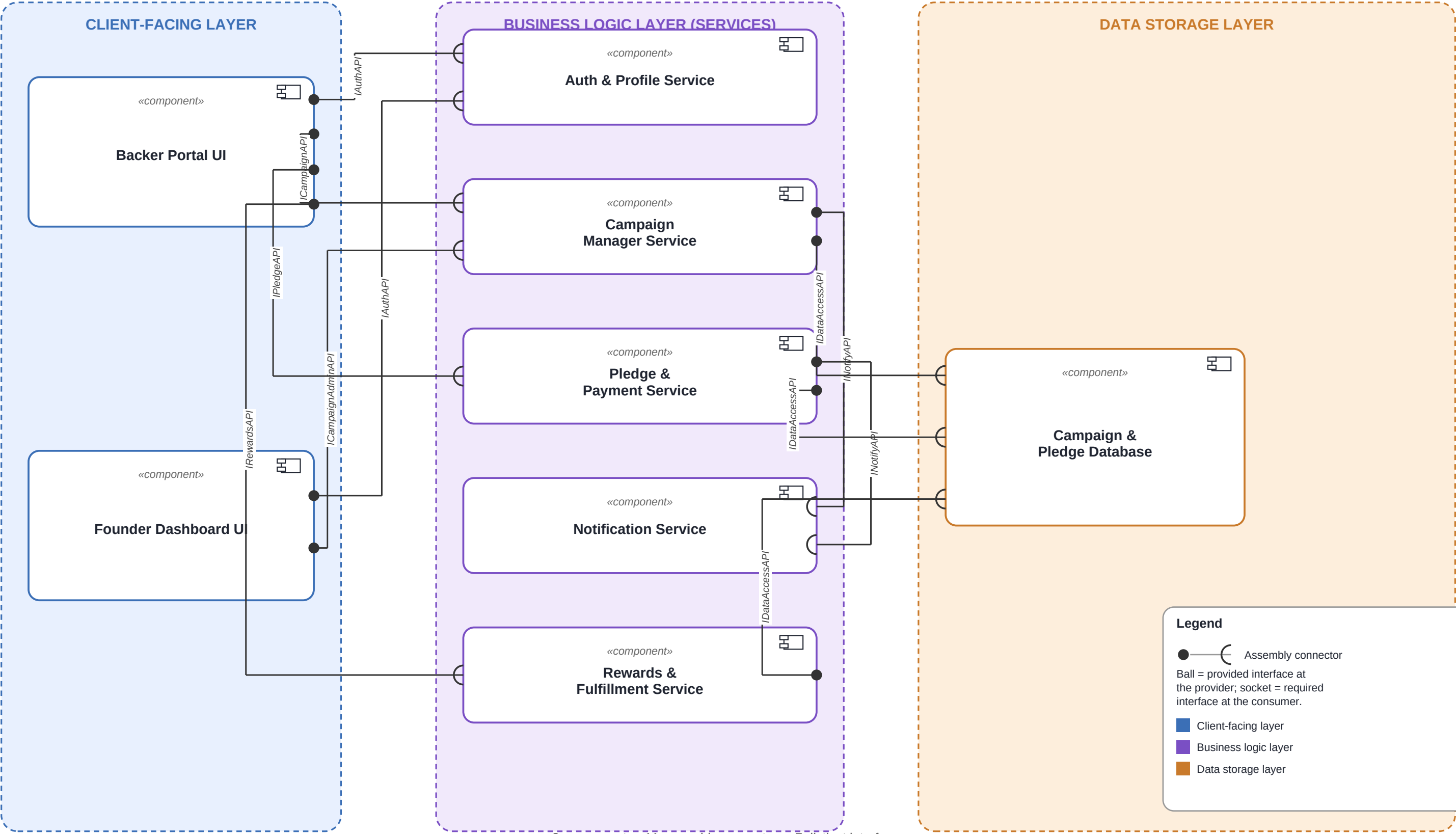


Student Startup Crowdfunding Platform

UML Component Diagram — Microservices Architecture



Lab 3 – Architectural Justification

Student Startup Crowdfunding Platform

Architecture Selection: “We chose Microservices Architecture for the Student Startup Crowdfunding Platform.”

Architectural Choice

The system is decomposed into five independently deployable services — Auth & Profile Service, Campaign Manager Service, Pledge & Payment Service, Notification Service, and Rewards & Fulfillment Service — fronted by two client components: the Backer Portal UI and the Founder Dashboard UI. Each service exposes a well-defined interface and owns its own logic, consistent with a Microservices architectural style.

Two Reasons for This Choice

1. Independent scaling for bursty, campaign-driven load: Pledge & Payment Service and Notification Service see sharp traffic spikes whenever a campaign goes viral or nears its funding deadline (the classic last-day pledge rush), while Auth and Rewards traffic stays comparatively flat. Microservices let the Pledge & Payment Service and Notification Service scale out independently during these windows without over-provisioning the rest of the system.

2. Fault isolation between core funding flow and secondary concerns: If the Notification Service (e.g., an email/SMS gateway) or the Rewards & Fulfillment Service becomes slow or unavailable, backers should still be able to browse campaigns and complete pledges. Because each service is an independently deployable unit with its own process boundary, a failure in Notification or Rewards does not cascade into the Campaign Manager or Pledge & Payment Service — unlike a monolithic layer where all functionality shares one failure domain.

Security Advantage

Centralizing authentication and authorization in a dedicated Auth & Profile Service means every other service, and both client UIs, enforce access control through one well-tested provided interface (IAuthAPI) instead of re-implementing checks locally. This narrows the attack surface, lets role-based permissions (Backer vs. Founder) be audited and updated in a single place, and prevents a compromise in one service from directly exposing pledge or payment data, since services communicate only through explicit, scoped interfaces rather than a shared session layer.

Performance Benefit

Because the Campaign Manager Service and Pledge & Payment Service are separate services with their own data access paths to the shared database, read-heavy campaign browsing (backers discovering and comparing campaigns) does not contend with write-heavy pledge and payment transactions on the same execution path, as it would in a single-layer monolith. The Notification Service can also be scaled and optimized independently for its bursty, fan-out messaging workload, keeping browsing latency low even during peak periods such as a campaign going viral or approaching its deadline.